



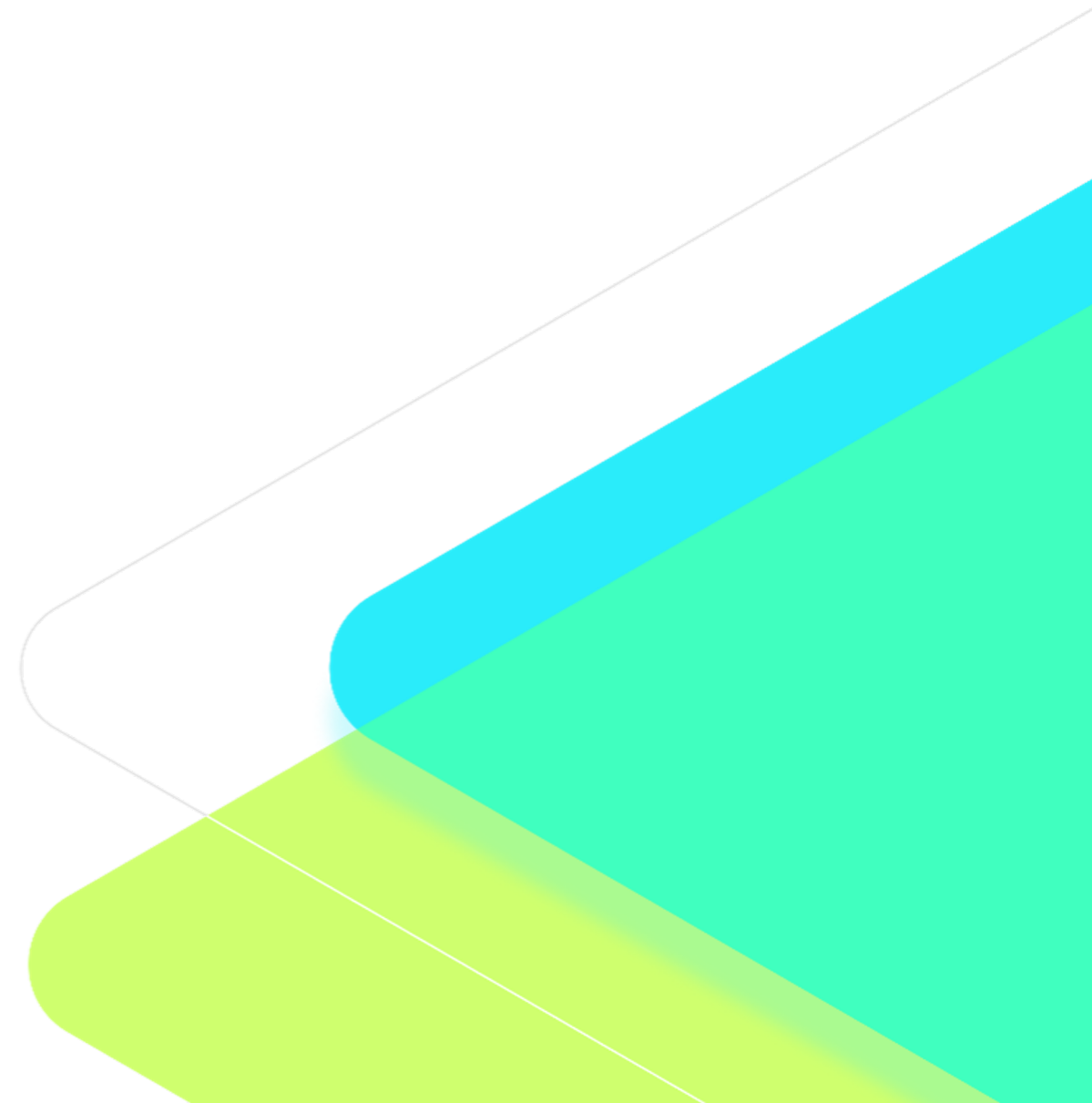
Retrievers and advanced retrieval strategies in LangChain



Deepali Srivastava

Author

[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)



Understanding the retriever abstraction in LangChain



What is a retriever in LangChain?



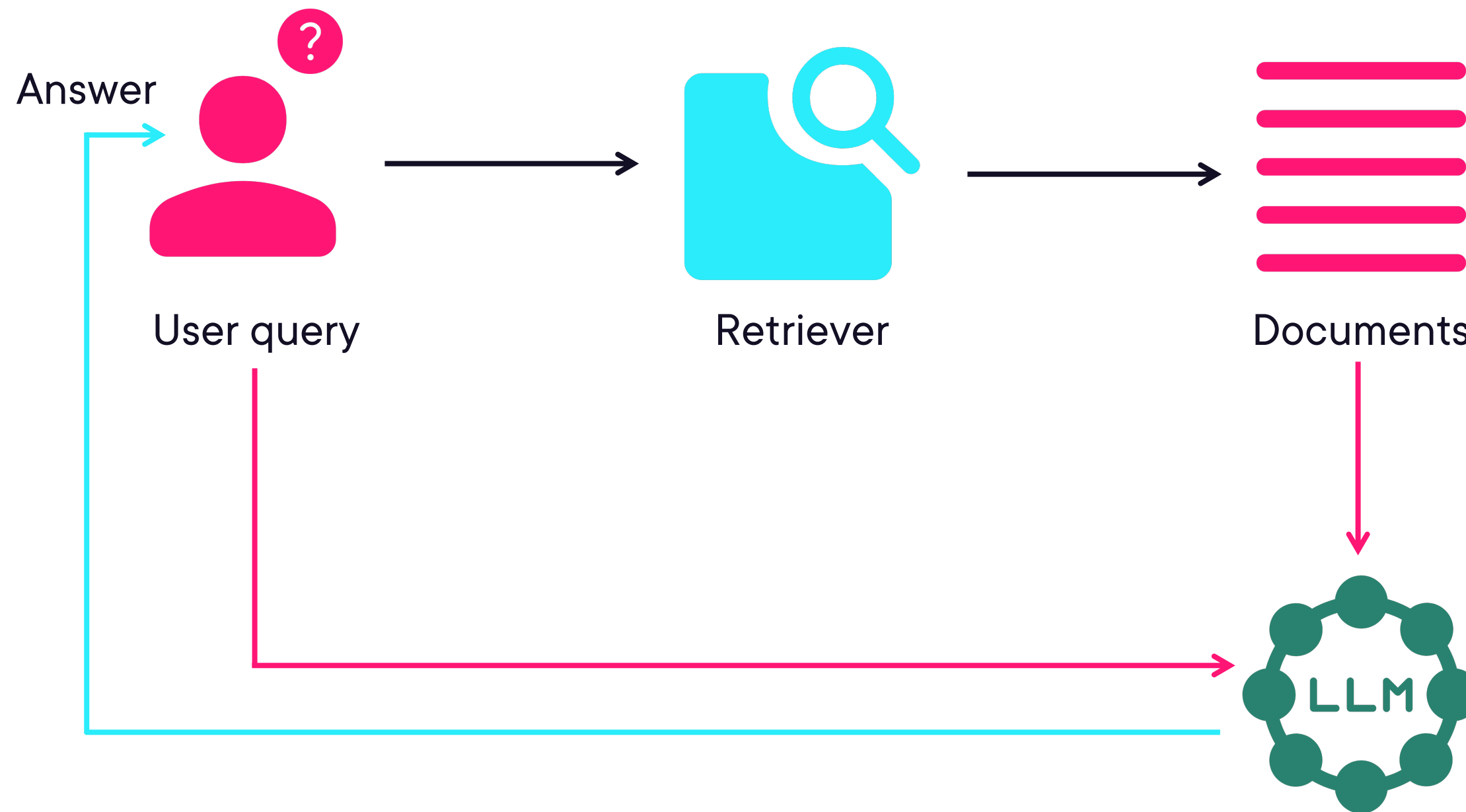
Retriever works across any data source

Not tied to any specific
data source

Can fetch data from vector
stores, databases, memory,
or APIs



Role of retriever in RAG systems



Better context leads to better answers

Retrieval quality is critical and can be improved using different strategies



Retriever provides a standard interface

```
docs = retriever.invoke(query)
```

Input is a query and output is a list of documents

Same usage regardless of underlying data source

Hides complexity and enables easy switching between sources



Retrievers are runnables in LangChain

Can be called
using `.invoke()`

They can be chained
and composed within
LangChain pipelines



Types of retrievers in LangChain



Source retrievers

- Data access layer
- Responsible for fetching documents
- **Examples:** VectorStoreRetriever, WikipediaRetriever



Strategy retrievers

- Enhancement layer
- Wrap a base retriever and improve retrieval process
- **Examples:** MultiQueryRetriever, ContextualCompressionRetriever



How to create a retriever in LangChain

Create it from a vector store

Use built-in retrievers

Wrap existing systems

Use advanced retrievers



Creating retriever from a vector store

```
from langchain_community.vectorstores import FAISS  
  
vectorstore = FAISS.from_documents(documents, embeddings)  
  
retriever = vectorstore.as_retriever()  
  
documents = retriever.invoke("user query")
```

Most common approach

Uses embeddings and similarity search



Using built-in retrievers

Wikipedia Retriever

```
from langchain_community.retrievers import WikipediaRetriever
retriever = WikipediaRetriever(top_k_results=3)
docs = retriever.invoke("user query")
```

BM25 Retriever

```
from langchain_community.retrievers import BM25Retriever
retriever = BM25Retriever.from_documents(docs, k=2)
docs = retriever.invoke("user query")
```

No embeddings or vector store required

Works with APIs or keyword-based search



Wrapping custom logic as a retriever

```
from langchain_core.retrievers import BaseRetriever

class MyCustomRetriever(BaseRetriever):

    def _get_relevant_documents(self, query: str) -> List[Document]:
        # Your custom logic goes here
        return documents

retriever = MyCustomRetriever()
docs = retriever.invoke("What is the status?")
```

Works with custom APIs or databases

Must follow standard interface



Advanced retrievers

```
from langchain_classic.retrievers.multi_query import MultiQueryRetriever

# Base retriever
base_retriever = vectorstore.as_retriever()

# MultiQuery retriever (LLM-powered)
advanced_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=llm
)
```

Built on top of an existing retriever

Improve the quality of retrieval



Implementing different types of retrievers in LangChain



Demo

Implementing BM25 retriever for keyword-based search



Demo

Using Wikipedia retriever for external knowledge retrieval



Demo

Creating a custom retriever



Demo

Creating and using a vector store retriever for semantic search



Improving recall with multi-query retrieval



Limitations of single-query retrieval

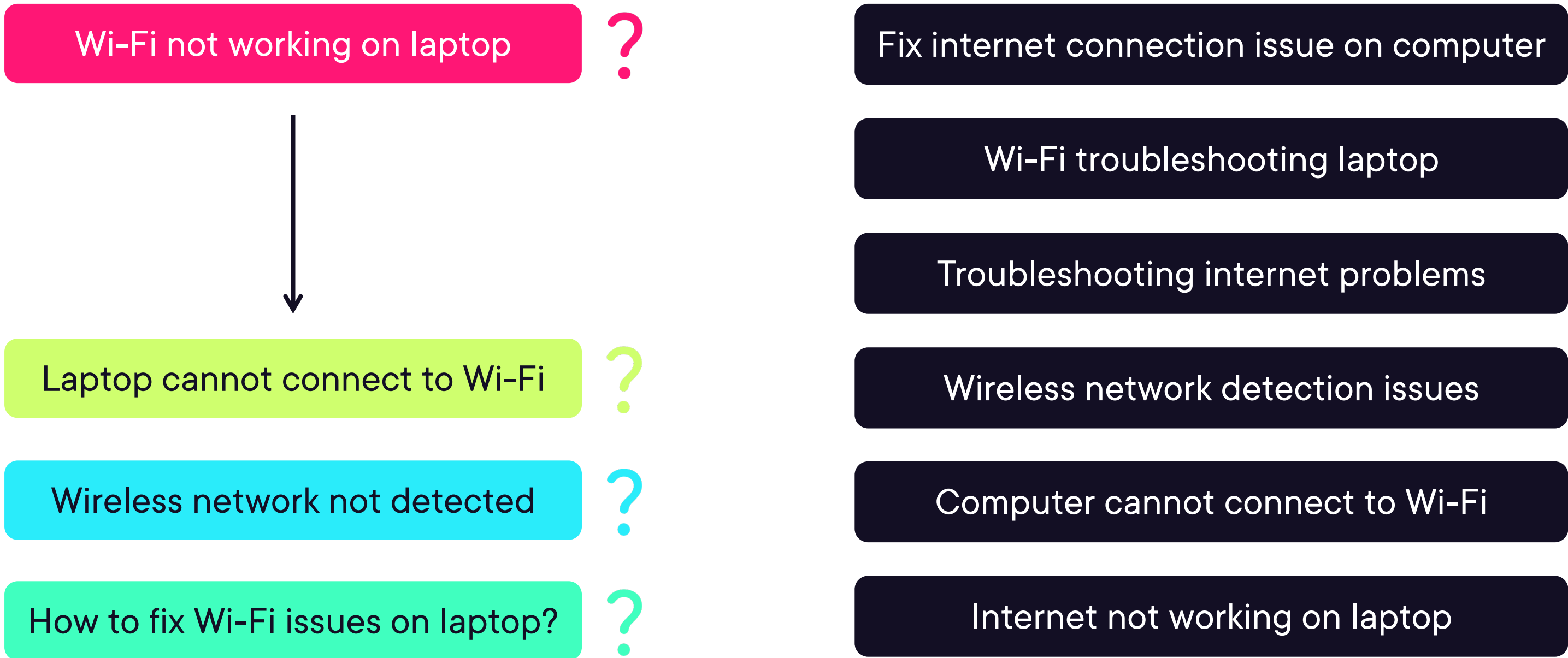
Results depend
on how the query
is phrased

Same information
can appear in
different forms

Semantic search
helps, but is
not perfect



Improving retrieval with multiple queries

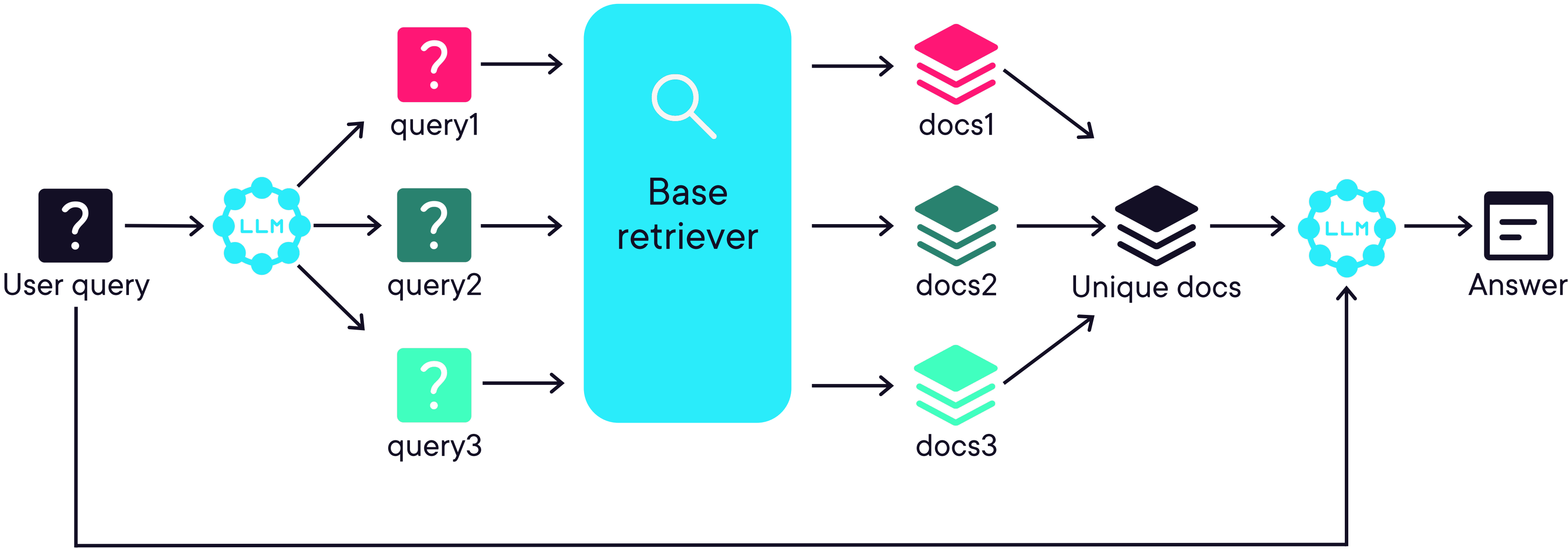


Multiple perspectives of the same question

Documents



How a multi-query retriever works



Multi-query retrieval: Benefits and tradeoffs

Improves recall
and coverage

May reduce precision

Higher cost
and latency

Best for large data and
ambiguous queries

Avoid for small data
or fast systems



Demo

Multi-query retrieval using MultiQueryRetriever



Demo

Building multi-query retrieval using LCEL (LangChain expression language)



Improving precision with contextual compression



Reducing noise in retrieval



Irrelevant content leads to worse answers and higher cost



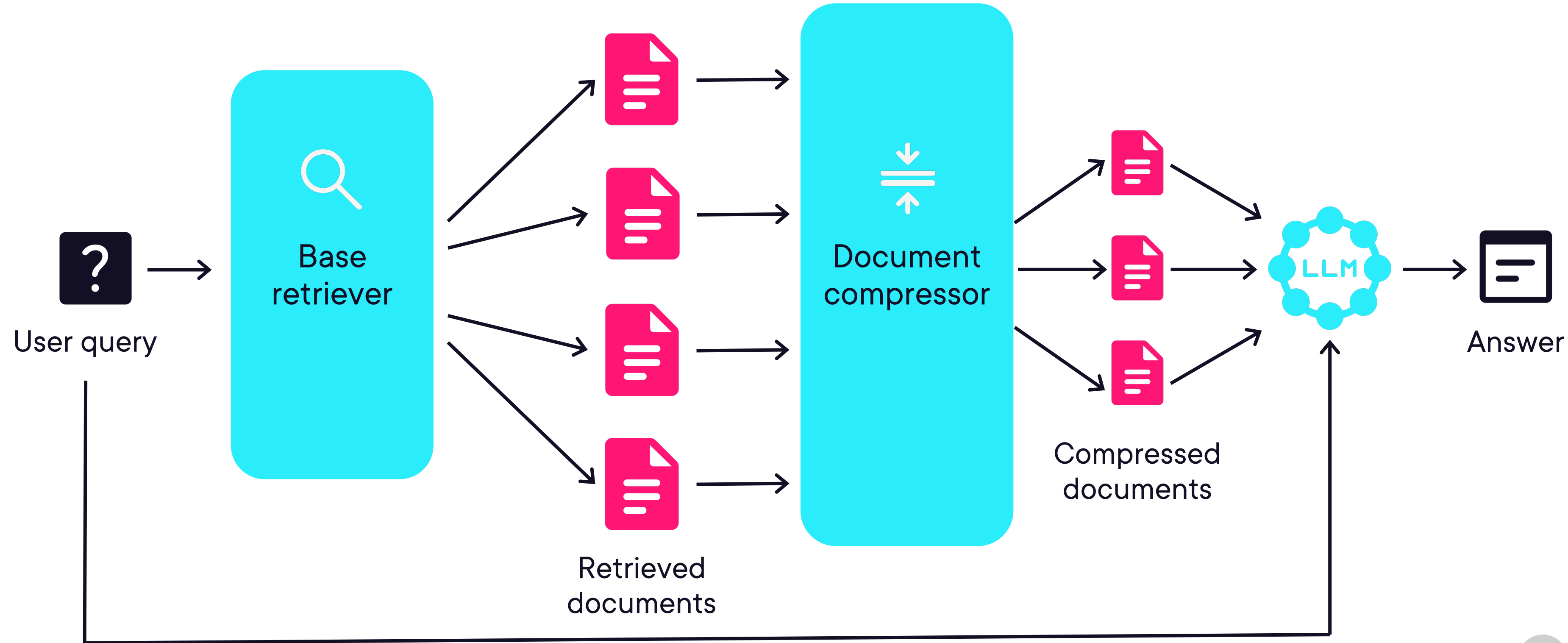
Contextual compression keeps only relevant parts



Document-level removes documents, span-level trims text within documents



How contextual compression works



Types of compressors in LangChain



LLM-based compressor

Understands context, extracts relevant parts



Embedding-based filters

Use similarity, faster but less precise



Keyword-based filters

Exact matches, very fast but no context



Benefits and tradeoffs of contextual compression

Higher precision and
reduced noise

Better answers and
lower token usage

Extra LLM call adds
latency and cost

Tradeoff between
quality and speed



Demo

Using ContextualCompressionRetriever



Demo

Contextual compression using LCEL



Evaluating retrieval systems



How to judge retriever performance

Are the retrieved documents actually relevant?

Did you miss any important information?

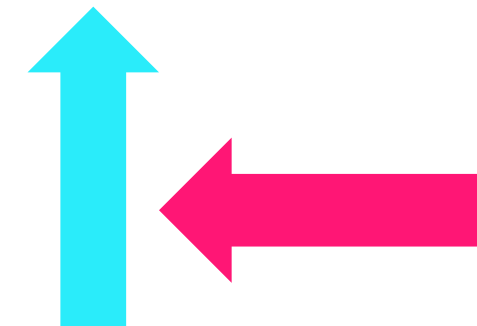
Are the most useful documents appearing at the top?



Recall and precision

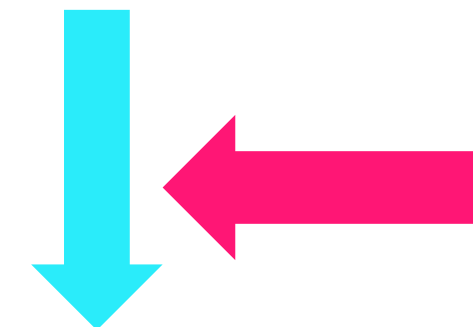
How many relevant documents appear in the top K results?

Recall@K = $\frac{\text{Number of relevant documents in the top K results}}{\text{Total number of relevant documents}}$



How many of the top K results are actually relevant?

Precision@K = $\frac{\text{Number of relevant documents in the top K results}}{\text{Total number of retrieved results (K)}}$



Recall and precision

How many relevant documents appear in the top K results?

Recall@K = $\frac{\text{Number of relevant documents in the top K results}}{\text{Total number of relevant documents}}$



How many of the top K results are actually relevant?

Precision@K = $\frac{\text{Number of relevant documents in the top K results}}{\text{Total number of retrieved results (K)}}$



Balancing precision and recall with F1 score

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean of precision and recall

Example 1: Balanced case

Precision = 0.8, Recall = 0.8
F1 = 0.8

Good balance → High F1

Example 2: Imbalanced case

Precision = 0.9, Recall = 0.1
F1 = 0.18

Poor balance → Low F1

Example 3: Opposite imbalance

Precision = 0.2, Recall = 0.9
F1 = 0.33

Poor balance → Low F1



Why ranking matters in retrieval



Top results matter most in RAG



Relevant documents may not appear at the top



You need to evaluate ranking quality



Mean reciprocal rank (MRR)

$$\text{MRR} = \frac{1}{\text{Rank of first relevant document}}$$

If the first relevant document is at:

- Rank 1 → MRR score = 1
- Rank 2 → MRR score = 0.5
- Rank 3 → MRR score = 0.33

Higher MRR means better ranking of the first relevant document



Tuning retrievers based on evaluation

Tune when results
are not satisfactory

Irrelevant answers or
missing information

Repetitive or redundant context

Adjust parameters to
improve performance



Tuning parameters in retrieval systems



k (number of results): Increases recall, may reduce precision



Multi-query: improves recall, increases cost



Compression: improves precision, may lose some information



MMR: improves ranking diversity, reduces redundancy



Similarity threshold: improves precision by filtering weak matches



Tuning based on evaluation metrics

If recall is low

- Increase k
- Use multi-query

If precision is low

- Use compression
- Use similarity thresholds

If ranking is poor

- Use MMR
- Improve result ordering



Trade-offs in retrieval systems



Trade-offs between speed, cost, and accuracy



A retrieval system must balance speed, cost, and accuracy



Increasing accuracy increases latency and cost



Improving speed may reduce accuracy



Reducing cost can impact retrieval quality



Retrieval strategies across different use cases



Customer support chatbot

Fast responses with a simple, lightweight pipeline



Medical or research assistant

High accuracy using advanced retrieval techniques



Enterprise knowledge base

Balanced approach using moderate settings



Design-level tradeoffs in retrieval systems



Vector store choice affects scalability, cost, and performance



Indexing strategy impacts speed vs. accuracy



Chunk size affects precision vs. context



Embedding model choice impacts accuracy, storage, and latency



Metadata filtering improves retrieval quality but increases complexity



Choosing the right retriever

Retrieval design is about making the right tradeoffs, not just applying techniques

The best approach depends on what matters most for the specific use case

